

Bancos de Dados Não-Relacionais

Aula 02: Teoremas CAP & PACELC e Paradigma BASE vs ACID

A presentation by

Prof. Me. Reginaldo Fernandes @ Instituto Federal do Ceará – Campus Tauá Tecnologia em Análise e Desenvolvimento de Sistemas

Agenda do Encontro

O Dilema dos Sistemas Distribuídos

O Teorema CAP

O Teorema PACELC

Duelo de Paradigmas: ACID vs BASE

Taxonomia NoSQL & Prática Hands-On

O Dilema dos Sistemas Distribuídos

De um Único Servidor para Clusters Distribuídos

O Mundo Relacional Clássico (Aula 01)

- Na Aula 01, vimos que bancos relacionais operam tradicionalmente sob **Escala Vertical (Scale-Up)**.
- Todas as leituras e escritas ocorrem na **mesma CPU e na mesma memória RAM**.
- A consistência é trivial: uma única máquina centralizada é dona da verdade absoluta.
- **Problema:** Teto físico de hardware e custo exponencial para sustentar Big Data.

A Realidade NoSQL: O Cluster Distribuído

- Para escalar horizontalmente (**Scale-Out**), particionamos dados entre dezenas ou centenas de servidores.
- **Novo Desafio Arquitetural:** Os nós precisam se comunicar continuamente através de placas de rede e roteadores.
- **Falhas Parciais:** Diferente de um único servidor que cai por inteiro, no cluster um nó pode falhar enquanto outros continuam ativos.

As 8 Falácias da Computação Distribuída

Deutsch & Gosling (Sun Microsystems, 1994)

- Todo desenvolvedor júnior comete suposições ingênuas que quebram em produção:
- 1. “A rede é confiável.” (Falso: fibras rompem, switches travam).
- 2. “A latência é zero.” (Falso: viajar entre SP e Fortaleza consome tempo físico).
- 3. “A largura de banda é infinita.”
- 4. “A rede é segura.”

As Falácias Estruturais

- 5. “A topologia da rede nunca muda.” (Nós entram e saem).
- 6. “Existe apenas um administrador.”
- 7. “O custo de transporte é zero.”
- 8. “A rede é homogênea.”
- **Impacto nos Bancos:** Como garantir que um cliente no Ceará e outro em São Paulo vejam o mesmo saldo ao mesmo tempo?

Cenário Prático: A Black Friday e o Saldo Bancário

O Drama do Arquiteto de Software

- Imagine as 00:00 da Black Friday: 100.000 requisições por segundo.
- Um cliente em Tauá possui R\$ 500,00 na conta e tenta fazer um PIX de R\$ 500,00 simultaneamente a uma compra no cartão.
- Ao mesmo tempo, um rompimento de fibra óptica isola o data center de Fortaleza do data center de São Paulo.

A Encruzilhada Operacional

- **Opção 1 (Prezar pela Verdade):** Bloquear as operações na região isolada até a rede voltar. O usuário recebe erro 503, mas o banco não perde dinheiro.
- **Opção 2 (Prezar pela Resposta):** Aceitar ambas as operações localmente. O usuário fica feliz na hora, mas gerou R\$ 500,00 de **gasto duplo** (double-spending).
- Esta escolha é o coração do Teorema CAP!

O Conceito Formal de Consistência (Linearizabilidade)

Consistência Forte / Linearizabilidade

- No contexto de sistemas distribuídos, **Consistência (C)** significa **Linearizabilidade** (Herlihy & Wing, 1990).
- Uma vez que uma escrita de dado $x = v$ for confirmada com sucesso ao cliente:
- **Qualquer leitura subsequente** realizada em qualquer nó do cluster deve obrigatoriamente retornar o valor v ou mais recente.

O Custo da Garantia

- Para assegurar Linearizabilidade, os nós precisam coordenar seus relógios e travar registros via **protocolos de consenso síncronos**.
- Se a rede romper ou atrasar, a coordenação fica impossível.
- **Conclusão:** A física impõe limites intransponíveis entre velocidade e coerência.

O Teorema CAP

Origem Histórica e Formulação Formal do Teorema CAP

Eric Brewer (ACM PODC, 2000)

- Proposto como uma conjectura pelo professor Eric Brewer (UC Berkeley) em 2000.
- Provado formalmente como teorema matemático em 2002 por **Seth Gilbert e Nancy Lynch** (MIT).
- Tornou-se o alicerce fundamental para a compreensão de todos os bancos de dados modernos.

O Enunciado Fundamental

- Em qualquer sistema distribuído de dados sujeito a comunicação assíncrona por rede:
- **É matematicamente impossível garantir simultaneamente:**
 - 1 Consistência (Consistency);
 - 2 Disponibilidade (Availability);
 - 3 Tolerância a Partições (Partition Tolerance).

As 3 Propriedades do CAP em Detalhes Rigorosos

C: Consistência (Linearizabilidade)

- Todo nó responde exatamente com o dado mais recente ou falha explicitamente.
- Comportamento idêntico a um sistema de máquina única centralizada.

A: Disponibilidade (Availability)

- Todo nó ativo e sem pane física **deve responder com sucesso** a qualquer requisição recebida (sem retornar erro 500/503 ou timeout).

P: Tolerância a Partições (Partition Tolerance)

- Capacidade do sistema de continuar operando mesmo que mensagens entre os nós sejam perdidas, atrasadas ou descartadas.
- A partição ocorre fisicamente quando um cabo rompe ou roteador falha, dividindo o cluster em dois ou mais subconjuntos isolados (**Split-Brain**).

Desmistificando o CAP: Por que “Escolha 2 de 3” é Falso?

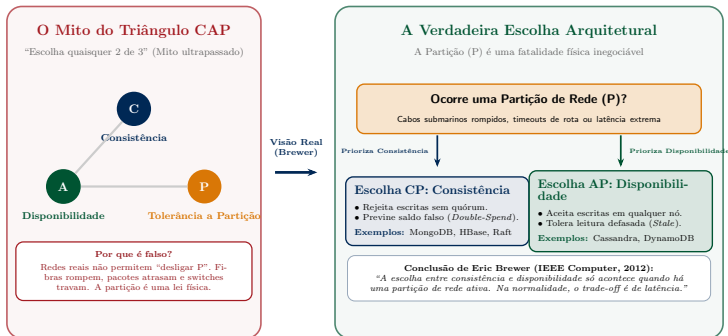
A Visão Antiga (O Triângulo Equivocado)

- Muitos tutoriais de internet dizem erroneamente: “Basta escolher CA, CP ou AP”.
- Afirmam que bancos como PostgreSQL ou MySQL seriam “CA” (Consistentes e Disponíveis).
- **O Erro Fatal:** Na internet e em redes de computadores, a Partição de Rede (P) **NUNCA** é **uma escolha**. Ela é um fato físico inegociável.

A Realidade (Brewer, 2012)

- “CAP Twelve Years Later: How the Rules Have Changed” (Eric Brewer, IEEE Computer 2012):
- Se a rede for 100% perfeita e local (máquina única), você tem C e A.
- **Mas quando ocorre uma partição de rede**, você é matematicamente forçado a escolher entre:
- **CP:** Preservar a consistência e sacrificar a disponibilidade; OU
- **AP:** Preservar a disponibilidade e sacrificar a consistência.

O Teorema CAP na Prática: Mito vs. Realidade



Regra de Ouro: Em sistemas distribuídos, a partição é inevitável. Sua única escolha é como agir durante a falha: CP ou AP.

A Regra de Quórum em Clusters de N Nós

A Matemática da Maioria Absoluta

- Para evitar que duas partes isoladas do cluster gravem dados conflitantes (*Split-Brain*), usa-se o conceito de **Quórum**:

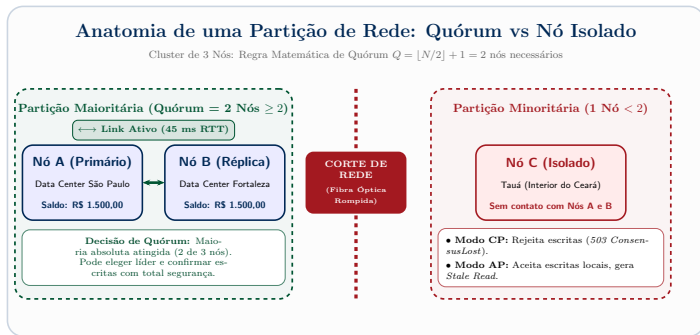
$$\text{Quórum Mínimo} = \left\lfloor \frac{N}{2} \right\rfloor + 1$$

- Onde N é o número total de nós votantes do cluster.
- Para $N = 3$ nós: Quórum = $1 + 1 = 2$ nós.
- Para $N = 5$ nós: Quórum = $2 + 1 = 3$ nós.

Por que o Número de Nós deve ser Ímpar?

- Se tivermos $N = 4$ nós e a rede dividir ao meio (2 e 2), **nenhum lado tem maioria!** O cluster inteiro para.
- Com nós ímpares (3, 5, 7), qualquer divisão binária garante que **apenas um lado** terá a maioria estrita.
- O lado maioritário continua operando com segurança. O lado minoritário entra em modo de proteção.

Anatomia do Quórum e Partição de Rede (Split-Brain)



Comportamento: O lado maioritário (A e B) mantém o quórum de escrita. O nó isolado C deve rejeitar escritas em CP ou aceitar divergência em AP.

Classificação CAP dos Principais Bancos de Dados

Bancos do Tipo CP (Consistência Estrita)

- **MongoDB (padrão):** Usa conjuntos de réplicas (Replica Sets). Se o primário for isolado da maioria, ele é destituído e recusa escritas.
- **Apache HBase / Bigtable:** Foco absoluto em consistência via ZooKeeper/Raft.
- **Redis (modo Cluster com Quórum):** Rejeita escritas em partições sem mestre válido.

Bancos do Tipo AP (Alta Disponibilidade)

- **Apache Cassandra:** Arquitetura masterless (todos os nós são iguais). Permite escrita em qualquer nó disponível.
- **Amazon DynamoDB:** Criado especificamente para nunca recusar o carrinho de compras.
- **CouchDB:** Projetado para funcionar offline e sincronizar via HTTP quando a conexão voltar.

O Teorema PACELC

Por que o Teorema CAP é Insuficiente no Dia a Dia?

O Ponto Cego de Eric Brewer

- Em 2012, o pesquisador **Daniel J. Abadi** (Yale University) publicou um artigo seminal na IEEE:
- “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story”.
- **A Pergunta Crítica:** O que acontece quando **NÃO** há partição de rede?

A Realidade da Nuvem Moderna

- Em provedores de ponta (AWS, Google Cloud, Azure), o cluster opera em estado normal durante **99,99% do tempo!**
- O CAP é completamente mudo sobre o comportamento do sistema durante 99,99% da sua vida útil.
- Mesmo sem falha de rede, existe um dilema feroz entre **Velocidade (Latência)** e **Coerência (Consistência)**.

A Formulação Mnemônica do Teorema PACELC

A Equação Arquitetural de Abadi

Se houver Partição de Rede (P):

- Escolha entre Disponibilidade (A) ou Consistência (C).

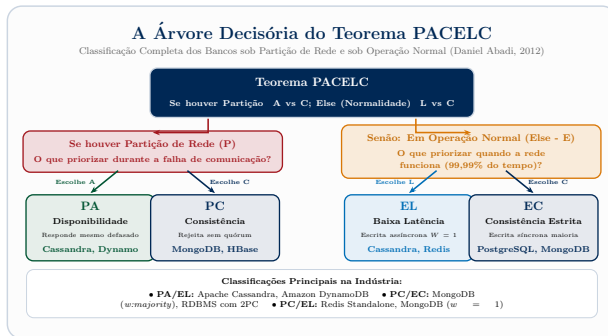
Senão (Else - E) em Operação Normal:

- Escolha entre Baixa Latência (L) ou Consistência (C).

As 4 Combinações Possíveis

- **PA/EL:** Prioriza alta disponibilidade na partição e latência ultrabaixa no dia a dia (Cassandra, DynamoDB).
- **PC/EC:** Prioriza consistência sempre, pagando o custo de esperar confirmações síncronas (MongoDB com maioria, RDBMS).
- **PC/EL:** Consistente na partição, mas otimizado para baixa latência local no dia a dia (Redis, MongoDB w=1).

A Árvore Decisória Completa do Teorema PACELC



Visão Prática: O Teorema PACELC mapeia o comportamento do banco tanto na crise (P) quanto no estado normal (Else).

Exemplo Analítico: O Custo Físico da Consistência

Cenário de Negócio Pareado

- Cluster com 3 nós em data centers distintos:
- Nó 1 (Local): $RTT_1 = 1,5$ ms (rack local).
- Nó 2 (Fortaleza): $RTT_2 = 45,0$ ms (regional).
- Nó 3 (Tauá): $RTT_3 = 58,0$ ms (interior).
- RTT (Round-Trip Time): tempo de ida e volta.

Cálculo Analítico de Latência

- Estratégia PA/EL (Escrita Local $W = 1$):

$$T_{EL} = RTT_1 + 0,3 = \mathbf{1,8 \text{ ms}}$$

- Estratégia PC/EC (Escrita Síncrona Maioria):

$$T_{EC} = \max_i(RTT_i) + 2,0 = \mathbf{60,0 \text{ ms}}$$

- Trade-off Físico: A consistência estrita custa $\approx 33,3\times$ mais tempo de resposta!

Demonstração Pareada em Python: Simulação PACELC

```
# RTT = Round-Trip Time em milissegundos
rtt_local = 1.5; rtt_fortaleza = 45.0; rtt_taua = 58.0

# PA/EL: Confirmacao imediata no primario local (W=1)
tempo_el = rtt_local + 0.3

# PC/EC: Confirmacao sincrona aguardando quorum
tempo_ec = max(rtt_local, rtt_fortaleza, rtt_taua) + 2.0

print(f"Latencia PC/EC (Consistencia Estrita) : {tempo_ec:.1f}
      ms")
print(f"Latencia PA/EL (Baixa Latencia W=1)   : {tempo_el:.1f}
      ms")
print(f"Aceleracao no modelo EL                  : {tempo_ec/
      tempo_el:.1f}x mais rapido!")
```

Tomada de Decisão: Para IoT/curtidas, escolha **EL**. Para financeiro, use **EC**.

Duelo de Paradigmas: ACID vs BASE

O Paradigma ACID: Filosofia Pessimista & Travas

Origem: Jim Gray (VLDB, 1981)

- Padrão ouro dos sistemas relacionais tradicionais (RDBMS):
- **Atomicidade:** A transação é indivisível (tudo ou nada).
- **Consistência:** Regras de negócio e chaves sempre válidas.
- **Isolamento:** Transações simultâneas não interferem.
- **Durabilidade:** Persistência permanente após commit.

O Gargalo da Abordagem Pessimista

- O isolamento estrito exige **travas de leitura e escrita** (Locks) e protocolos como **2PC** (Two-Phase Commit).
- Sob concorrência massiva, travas geram filas e **deadlocks**.
- **Filosofia:** *“Prefiro travar a requisição a correr o risco de gravar um dado imperfeito.”*

O Paradigma BASE: Filosofia Otimista & Alta Escala

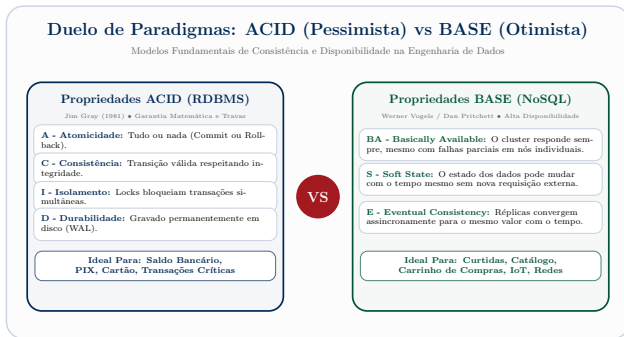
Origem: Werner Vogels & Dan Pritchett

- Criado para sustentar as gigantes da Web (Amazon, Google):
- **BA – Basically Available:** O sistema continua respondendo mesmo com falhas parciais em nós.
- **S – Soft State:** O estado dos dados pode mudar com o tempo mesmo sem nova requisição externa.
- **E – Eventual Consistency:** Réplicas convergem assincronamente para o mesmo valor com o tempo.

A Vantagem da Abordagem Otimista

- Elimina completamente as travas distribuídas bloqueantes.
- Permite vazão de centenas de milhares de escritas por segundo.
- **Filosofia:** “Prefiro responder em milissegundos e reconciliar os dados assincronamente em segundo plano.”

Duelo dos Paradigmas: ACID vs. BASE Lado a Lado



Diretriz Arquitetural: Não existe modelo superior. O bom engenheiro de software escolhe o paradigma de acordo com o domínio de negócio.

Como Funciona a Consistência Eventual na Prática?

O Algoritmo Last-Write-Wins (LWW)

- Cada escrita recebe um carimbo de tempo físico (timestamp).
- A versão com maior carimbo sobrescreve versões anteriores.
- **Risco Real:** Desvios de relógio (Clock Drift) entre servidores podem descartar escritas legítimas.

Vetores de Versão (Vector Clocks)

- Cada nó mantém contadores lógicos por registro.
- Identifica se uma alteração foi causal ou concorrente.
- **Caso Real (Amazon Dynamo):** O carrinho de compras une itens conflitantes em vez de apagar qualquer um!

Matriz Decisória de Negócio: Onde Aplicar Cada Paradigma

Domínio de Negócio	Paradigma	Justificativa Arquitetural
Transferência PIX	ACID / CP	O saldo não tolera inconsistência. Gasto duplo gera rombo financeiro.
Curtidas no TikTok	BASE / AP	Perder pequenas contagens em milhões de visualizações é aceitável.
Carrinho de Compras	BASE / AP	Rejeitar o cliente perde a venda. Melhor reconciliar itens depois.
Vagas SISU / Concursos	ACID / CP	Disputas por fração de segundo exigem isolamento estrito.

Taxonomia NoSQL & Prática Hands-On

Os 4 Modelos NoSQL & Bancos Vetoriais

Modelos Orientados a Agregados

- **Chave-Valor (Redis, DynamoDB):** Dicionários gigantes em memória. Complexidade $O(1)$. Foco em cache e latência (**PA/EL** ou **PC/EL**).
- **Documentos (MongoDB, Couchbase):** JSON/BSON com esquemas flexíveis e aninhamento. Foco em aplicações web e catálogos (**PC/EC**).
- **Família de Colunas (Cassandra, ScyllaDB):** Colunas esparsas distribuídas em anel masterless. Escrita massiva (**PA/EL**).

Grafos & Bancos Vetoriais

- **Orientados a Grafos (Neo4j):** Foco na riqueza de relacionamentos e travessia rápida de caminhos (**PC/EC**).
- **Bancos Vetoriais (ChromaDB, Pinecone):** Vetores de alta dimensão (*embeddings*) para busca semântica e IA Generativa (RAG).

Taxonomia NoSQL na Matriz CAP / PACELC

Taxonomia NoSQL & Posicionamento na Matriz PACELC / CAP

Como os 4 Modelos Fundamentais e os Bancos Vetoriais Escolhem seus Trade-offs

1. Modelo Chave-Valor (Key-Value)

Exemplos: Redis, Memcached, Amazon DynamoDB

- **Estrutura:** Tabela hash com complexidade $O(1)$.
- **PACELC:** PA/EL (Dynamo) ou PC/EL (Redis).
- **Cenário:** Caching, Sessões Web, Filas de Mensagens.

2. Modelo de Documentos (Document Store)

Exemplos: MongoDB, Couchbase, Google Cloud Firestore

- **Estrutura:** Documentos hierárquicos JSON/BSON flexíveis.
- **PACELC:** PC/EC (*majority*) ou ajustável por escrita.
- **Cenário:** E-commerce, Catálogos de Produtos, APIs Web.

3. Família de Colunas (Wide-Column)

Exemplos: Apache Cassandra, ScyllaDB, Apache HBase

- **Estrutura:** Linhas esparsas particionadas em anel *master-less*.
- **PACELC:** PA/EL (Alta Disponibilidade e Escrita Massiva).
- **Cenário:** Séries Temporais, Telemetria IoT e Métricas.

4. Orientado a Grafos (Graph Database)

Exemplos: Neo4j, Amazon Neptune, Memgraph

- **Estrutura:** Vértices (nós), arestas direcionadas e índices.
- **PACELC:** PC/EC (Foco na integridade e travessia rápida).
- **Cenário:** Redes Sociais, Detecção de Fraudes, Logística.

Modernização: Bancos Vetoriais (Vector Databases) – ChromaDB, Pinecone, Milvus, Qdrant:

Otimizados para busca por similaridade de cosseno (k -NN) sobre representações vetoriais de IA Generativa (RAG).

Síntese Arquitetural: Cada modelo NoSQL foi desenhado para resolver um tipo específico de acesso e trade-off.

MongoDB Prático: Níveis de Garantia de Escrita (Write Concern)

Ajustando o Teorema PACELC diretamente no código de aplicação:

```
// 1. Alta Performance / Baixa Latencia (Modo PA/EL):
db.sensores.insertOne(
  { sensor: "TAU-01", temp: 31.2 },
  { writeConcern: { w: 1, wtimeout: 1000 } }
);

// 2. Alta Seguranca e Consistencia Financeira (Modo PC/EC):
db.transferencias.insertOne(
  { de: "CTA-100", para: "CTA-200", valor: 1500.00 },
  { writeConcern: { w: "majority", j: true, wtimeout: 5000 } }
);
```

Nota de Engenharia: O parâmetro w: "majority" impede que alterações sejam revertidas por eleições de novo líder.

MongoDB Prático: Níveis de Isolamento de Leitura (Read Concern)

Controlando se sua consulta pode ou não ler dados defasados (Stale Reads):

```
// 1. Leitura Local (Rápida, mas com risco de Stale Read):  
db.pedidos.find({ status: "pendente" }).readConcern("local");  
  
// 2. Leitura com Quorum Garantido (Majority Read Concern):  
db.pedidos.find({ id_pedido: "PED-2026-09" }).readConcern("majority");  
  
// 3. Leitura Linearizável (Linearizable):  
db.contas.find({ id_cliente: 42 }).readConcern("linearizable");
```


Roteiro do Laboratório Hands-On no Terminal

O Script Didático da Aula

- Desenvolvido especificamente para esta aula: `demonstracao_aula02.py`.
- Simula um cluster de 3 nós com injeção de rompimento de rede em tempo real.
- Demonstra na prática:
 - Bloqueio de escritas sem quórum (CP);
 - Escrita local e leitura defasada (AP);
 - Reconciliação via *Last-Write-Wins*.

Como Executar no Laboratório

- Abra o terminal na pasta da aula e execute:
- `python3`
`codigo/demonstracao_aula02.py`
- Navegue pelo menu interativo (opções 1 a 5).
- Também disponível como Jupyter Notebook interativo:
- `codigo/demonstracao_aula02.ipynb`

Próximo Encontro & Atividades de Fixação

Sábado Letivo (10/10/2026) – Aula 03

- **Atenção:** Conforme o cronograma pedagógico, sábado letivo **não tem conteúdo novo**.
- **Oficina Prática de DevOps:** Orquestração com Docker e Docker Compose para subir instâncias de Redis e MongoDB em segundos.
- Resolução assistida da Lista 1 no laboratório.

Lista de Exercícios 1 (Bloco 2)

- Responder às questões 7 a 12 no `README.md` da Aula 02:
 - Análise de partição em 3 e 4 nós;
 - Trade-offs de Latência vs Consistência no PACELC;
 - Comparação de cenários do mundo real (PIX vs TikTok);
 - Uso de Write Concern no MongoDB.

Dúvidas & Discussão

IFCE Campus Tauá – ADS16

reginaldo.fernandes@ifce.edu.br